

Proyecto Front End

Delegar la escritura de código en la inteligencia artificial no es sinónimo de abandonar la responsabilidad técnica; por el contrario, exige una evolución del profesional hacia el rol de director de orquesta o arquitecto estratégico. La verdadera eficiencia en el desarrollo moderno no reside en la generación masiva de líneas de código, sino en la capacidad de estructurar un pensamiento lógico previo que guíe a la IA bajo principios de **Clean Architecture** y metodologías rigurosas como TDD (Test-Driven Development). Este enfoque transforma el proceso creativo en un ejercicio de validación constante donde los conceptos de negocio y las reglas de arquitectura predominan sobre la sintaxis. Dominar esta sinergia permite reducir drásticamente los tiempos de configuración y desarrollo de infraestructuras complejas —como el setup de un entorno robusto con React, TypeScript y herramientas de testing—, permitiendo que el foco se desplace hacia la resolución de problemas de alto valor, como la optimización de la experiencia de usuario y la seguridad operativa. Al implementar una estrategia de 'Software Engineering asistido', se establece un flujo de trabajo donde la calidad es un requerimiento mandatorio desde el minuto cero, garantizando que el producto resultante sea modular, escalable y, sobre todo, mantenible a largo plazo.

Arquitectura y Metodología de Implementación

La Regla del Alcance (Scope Rule)

La organización del proyecto se rige por la lógica de funcionalidad sobre la técnica. Si un componente o utilidad tiene un uso transversal (como tipos globales o funciones de formateo), se ubica en el Global Scope (Source/Share). Si su funcionalidad es específica, permanece en el Local Scope de su feature. Esta estructura facilita el Lazy Loading y optimiza el rendimiento del bundle al cargar solo lo estrictamente necesario para cada vista.

Ciclo de Vida Estandarizado con TDD

El desarrollo no comienza en el componente, sino en el test. Bajo el ciclo de Red-Green-Refactor, se define primero el comportamiento esperado mediante pruebas unitarias que inicialmente fallan por la inexistencia del código. Una vez que la IA genera la implementación mínima necesaria para pasar a verde, se procede a la optimización, eliminando duplicidades y mejorando la legibilidad sin comprometer la funcionalidad ya validada.

Estrategias de Negocio y Lógica Computada

En el desarrollo de un motor de descuentos, se ha optado por el **Strategy Pattern** para desacoplar las reglas de negocio de la lógica del carrito. Esto permite que el sistema aplique descuentos por volumen (Bulk Discount) o por valor total de orden de forma secuencial y transparente. La utilización de un 'Discount Calculator' coordina estas estrategias, permitiendo que cada una actúe sobre el monto restante, garantizando precisión financiera en entornos de alta transaccionalidad.

Validación de Entorno y Quality Gates

Testing de Extremo a Extremo (E2E)

Mediante el uso de Page Objects en Playwright, se simula la interacción real del usuario, validando no solo la lógica, sino la persistencia en LocalStorage y el flujo completo de compra. Estos tests incluyen regresión visual para asegurar que refactorizaciones cosméticas no rompan la interfaz de usuario en diferentes navegadores.

Automatización con Husky y Quality Gates

La integridad del código se protege mediante hooks de Git que ejecutan procesos de **Linting**, Type-checking y cobertura de tests (Coverage) antes de cada commit o push. Se establecen umbrales (Thresholds) estrictos: un fallo en la cobertura mínima de lógica de negocio o un error de accesibilidad bloquea automáticamente la subida de código, asegurando que la deuda técnica se mantenga bajo control.

Observaciones Clave

- La eficiencia de la IA depende directamente de la claridad del System Prompt; definir el rol como Senior Architect reduce drásticamente las alucinaciones del modelo.
- Es crítico el uso de **as const** en TypeScript para que los valores de negocio no sean mutables y se conozcan en tiempo de compilación.
- La accesibilidad no es opcional: el uso de ARIA labels dinámicos y regiones 'Live' (Polite) garantiza que el software sea inclusivo desde su concepción.
- Se debe excluir sistemáticamente el directorio de tests en las configuraciones de build (tsconfig) para evitar errores de compilación innecesarios en producción.
- El monitoreo de errores a través de Sentry permite capturar fallos en tiempo real mediante el Error Boundary, vinculando cada incidencia al contexto del usuario.

Conclusión estratégica

La integración de inteligencia artificial en el ciclo de desarrollo bajo estándares de ingeniería senior redefine la competitividad empresarial. No se trata solo de producir software más rápido, sino de construir sistemas robustos que incluyan de serie **Quality Gates** automáticos, alta cobertura de pruebas y una arquitectura resiliente. Este enfoque permite que las organizaciones escalen sus productos digitales con la seguridad de que cada nueva iteración ha sido validada técnica y funcionalmente, minimizando el riesgo operativo y maximizando el retorno de inversión tecnológica.